29-cloud-computing

29. Cloud Computing

Level: 🟢 Beginner → 🟡 Intermediate
Jam: 8 (4 minggu × 2 sesi)
Prasyarat: Dasar programming (javascript/typescript), familiar dengan CLI
Output: App yang di-deploy ke cloud dengan CI/CD pipeline

Tujuan Pembelajaran

Setelah modul ini, kamu bisa: - Paham konsep cloud computing (IaaS/PaaS/SaaS) dan model deployment - Bedain cloud providers dan pilih yang sesuai kebutuhan - Deploy compute instance (EC2/Droplet) dan manage storage (S3/Spaces) - Pake CDN dan file upload handling - Nulis & deploy serverless functions (Cloudflare Workers, Vercel Edge) - Integrasi serverless database (Turso/Neon) - Manage environment variables dan secrets - Pake Infrastructure as Code (Terraform/Pulumi) - Setup CI/CD pipeline ke cloud pakai GitHub Actions - Monitor app yang udah di-deploy

Materi

Sesi	Topik	File
1	Cloud Basics: IaaS/PaaS/SaaS, Region/AZ, VPC, Provider Comparison	01-cloud-basics.md
2	Compute & Storage: EC2/Droplet, Auto Scaling, S3/Spaces, CDN, File Upload	02-compute-storage.md

Sesi	Topik	File
3	Serverless Functions: Cloudflare Workers, Vercel Edge, Railway, Serverless DB, Secrets	03-serverless-functions.md
4	IaC & Deploy: Terraform, Docker, CI/CD GitHub Actions, Monitoring	04-iac-deploy.md

Output Akhir Modul

> **App deployed on cloud with CI/CD** — REST API atau static site yang di-deploy ke DigitalOcean / Cloudflare, dengan pipeline otomatis dari GitHub

AI Prompt Exercises

Sepanjang modul, latihan pake AI: - “Explain the difference between IaaS, PaaS, and SaaS with examples” - “Generate a CloudFormation/Terraform script for a VPC with two subnets” - “Debug this serverless function: it times out on large requests” - “Compare pricing between AWS Lambda and Cloudflare Workers for 1M requests/month” - “Write a GitHub Actions workflow to deploy to DigitalOcean App Platform”

1. Cloud Basics: Models, Region, VPC & Provider Comparison

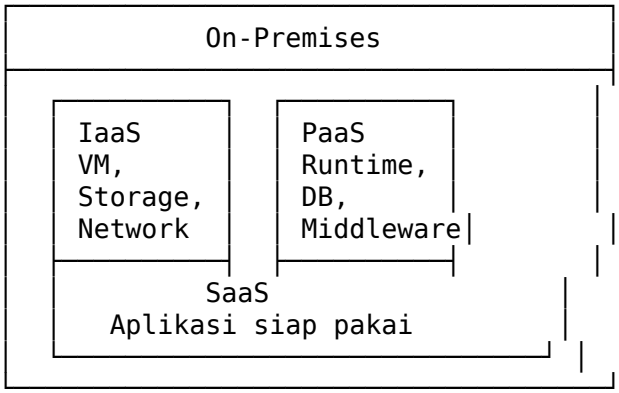
Konsep Cloud Computing

Cloud computing = deliver computing resources (server, storage, database, networking) over internet. Bayar sesuai pemakaian (pay-as-you-go).

Deployment Models

Model	Deskripsi	Contoh
Public Cloud	Provider punya infrastruktur, dipakai banyak customer	AWS, GCP, Azure, DigitalOcean
Private Cloud	Infrastruktur khusus 1 organisasi	OpenStack, VMware on-prem
Hybrid Cloud	Gabungan public + private	AWS Outposts, Azure Arc
Multi-cloud	Pake >1 public cloud provider	AWS + GCP

Service Models



Model	Kamu Manage	Provider Manage	Contoh
IaaS	OS, app, data	Server, network, storage fisik	AWS EC2, DigitalOcean Droplet
PaaS	App + data	OS, runtime, infra	Heroku, Railway, Vercel
SaaS	Hanya data	Semua	Google Workspace, Notion, Slack

Region & Availability Zone

- **Region**: Lokasi geografis data center (us-east-1, ap-southeast-1, ap-southeast-3)

- **Availability Zone (AZ)**: Data center terisolasi dalam 1 region (us-east-1a, us-east-1b)
- **Edge Location**: PoP (Point of Presence) untuk CDN — lebih dekat ke user

```
Region: ap-southeast-1 (Singapore)
├── AZ: ap-southeast-1a
├── AZ: ap-southeast-1b
└── AZ: ap-southeast-1c
        │
        └── Edge Locations: Jakarta, Manila, Bangkok, ...
```

Penting: Pilih region terdekat dengan target user untuk latency rendah.

VPC (Virtual Private Cloud)

VPC = jaringan virtual terisolasi di cloud. Subnet, firewall, routing kamu atur sendiri.

```
# DigitalOcean VPC — CLI
doctl vpc create --name "my-vpc" --region "sgp1" --ip-range "10.10.0.0/16"
```

Komponen VPC: - **Subnet**: Bagian IP range (public = internet access, private = internal) - **Route Table**: Atur lalu lintas jaringan - **Internet Gateway**: Gerbang ke internet - **NAT Gateway**: Private subnet akses internet (keluar aja) - **Security Group / Firewall**: Aturan allow/deny traffic

```
# Firewall rules — DigitalOcean
doctl compute firewall create \
  --name "web-firewall" \
  --inbound-rules "protocol:tcp,ports:80,address:0.0.0.0/0,protocol:tcp,ports:44
```

Cloud Providers Comparison

Fitur	AWS	GCP	Azure	DigitalOcean	Biznet Gio
Compute	EC2	Compute Engine	VM	Droplet	Gio VM
Serverless	Lambda	Cloud Functions	Azure Functions	Functions	—
Object Storage	S3	Cloud Storage	Blob	Spaces	Gio Object

Fitur	AWS	GCP	Azure	DigitalOcean	Biznet Gio
Managed DB	RDS	Cloud SQL	SQL DB	Managed DB	Gio DB
Container	ECS/EKS	GKE	AKS	DOKS	—
DNS + CDN	CloudFront + 53	Cloud CDN	Azure CDN	DO DNS + CF	—
Free Tier	12 bulan	90 hari (kredit)	12 bulan	$200 kredit 60hr	—
Harga VM (termurah)	~$8.5/bln	~$7/bln	~$7/bln	~$6/bln	~$5/bln
Learning Curve	Tinggi	Sedang	Tinggi	Rendah	Rendah

Kapan Pilih Mana?

- **AWS**: Feature lengkap, market leader, banyak belajar resource
- **GCP**: AI/ML bagus, network kenceng, BigQuery
- **Azure**: Enterprise, integrasi Microsoft
- **DigitalOcean**: Developer friendly, harga predictable, cocok UKM
- **Biznet Gio**: Local (Indonesia), latency rendah, support Indonesia

Cost Estimation

AWS Calculator

https://calculator.aws

DigitalOcean Pricing

```
# Cek harga Droplet
doctl compute size list

# Contoh: 1 Droplet basic ($6/bln) + 1 Spaces ($5/bln) + 1 Load Balancer ($12/bl
# Total: ~$23/bln
```

Free Tier Checklist

Provider	Free Tier
AWS	750 jam EC2 t2.micro/bln, 5GB S3, 1GB RDS — 12 bulan
GCP	$300 kredit — 90 hari, Always Free: f1-micro (1 CPU, 614MB)
Azure	$200 kredit — 30 hari, 750 jam B1s VM — 12 bulan
DigitalOcean	$200 kredit — 60 hari
Cloudflare	Free tier: Workers 100k req/hari, R2 10GB, Pages unlimited, D1 5GB

Latihan

1. **Bandingkan Provider**: Pilih 3 provider (misal AWS vs DO vs Cloudflare). Buat tabel perbandingan untuk use case: "deploy REST API + database + CDN". Hitung estimasi biaya per bulan. Tulis argumen kenapa pilih provider A.
2. **VPC Setup**: Pakai DigitalOcean atau AWS free tier. Buat VPC dengan 2 subnet (public + private). Deploy 1 Droplet/EC2 di public subnet. Akses SSH cuma dari IP kamu. Screenshot hasil.
3. **Region Simulasi**: Pakai tool `ping` atau `cloudping.com`. Test latency dari lokasi kamu ke 3 region (Singapore, California, London). Catat hasilnya. Hitung rata-rata latency.

```
# Contoh test ping ke AWS region
ping -c 10 ec2.ap-southeast-1.amazonaws.com
ping -c 10 ec2.us-west-1.amazonaws.com
ping -c 10 ec2.eu-west-2.amazonaws.com
```

4. **Free Tier Challenge**: Daftar 2 provider free tier. Deploy 1 static HTML page di masing-masing (AWS S3 static hosting + Cloudflare Pages atau DigitalOcean App Platform). Catat prosesnya dan perbedaan UX.

2. Compute & Storage: EC2/Droplet, Auto Scaling, Object Storage, CDN, File Upload

Compute Instance (VM di Cloud)

DigitalOcean Droplet

```
# Create Droplet via doctl
doctl compute droplet create \
```

```
  --name "web-server" \
  --region "sgp1" \
  --size "s-1vcpu-1gb" \
  --image "ubuntu-24-04-x64" \
  --ssh-keys "YOUR_SSH_KEY_ID"

# SSH masuk
ssh root@<droplet-ip>
```

AWS EC2

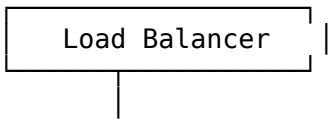
```
# Via AWS CLI
aws ec2 run-instances \
  --image-id ami-0c55b159cbfafe1f0 \
  --instance-type t2.micro \
  --key-name MyKeyPair \
  --security-group-ids sg-xxxx \
  --subnet-id subnet-xxxx

# SSH
ssh -i ~/.ssh/MyKeyPair.pem ec2-user@<public-ip>
```

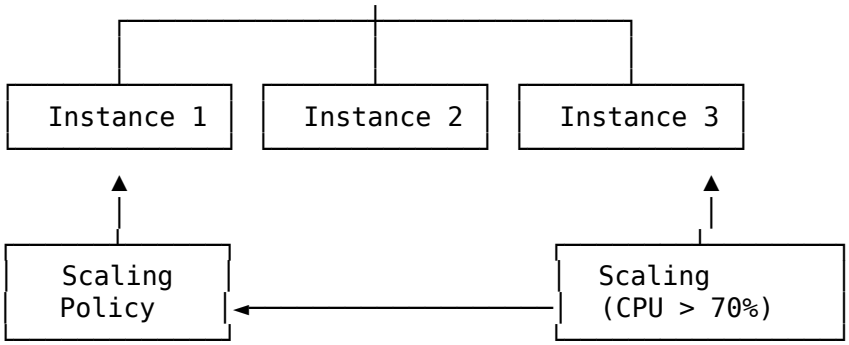
Setup Web Server (Ubuntu)

```
# install nginx
apt update && apt install -y nginx

# firewall
ufw allow 'Nginx Full'
ufw allow OpenSSH
ufw enable

# cek status
systemctl status nginx
curl http://localhost
```

Auto Scaling

Auto Scaling = otomatis nambah/ngurang instance berdasarkan traffic.





Basic Setup — DigitalOcean

```
# 1. Buat snapshot Droplet sebagai base image
doctl compute droplet-action snapshot <droplet-id> --snapshot-name "web-snapshot

# 2. Buat Load Balancer
doctl compute load-balancer create \
  --name "web-lb" \
  --region "sgp1" \
  --forwarding-rules "entry_protocol:http,entry_port:80,target_protocol:http,tar

# 3. Buat tag
doctl compute tag create "web-server"

# 4. Saat traffic naik, manual scale:
doctl compute droplet create \
  --name "web-server-2" \
  --region "sgp1" \
  --size "s-1vcpu-1gb" \
  --image "web-snapshot" \
  --tag-names "web-server"
```

Auto Scaling — AWS (ASG)

```
# Terraform — Auto Scaling Group
resource "aws_autoscaling_group" "web_asg" {
  name                = "web-asg"
  min_size            = 2
  max_size            = 10
  desired_capacity    = 2
  vpc_zone_identifier = ["subnet-xxx", "subnet-yyy"]

  launch_template {
```

```
    id      = aws_launch_template.web.id
    version = "$Latest"
  }
}

resource "aws_autoscaling_policy" "cpu_up" {
  name                   = "cpu-up"
  scaling_adjustment     = 1
  adjustment_type        = "ChangeInCapacity"
  autoscaling_group_name = aws_autoscaling_group.web_asg.name
}
```

Object Storage

AWS S3

```
# buat bucket
aws s3 mb s3://my-app-storage-123

# upload file
aws s3 cp ./profile.jpg s3://my-app-storage-123/uploads/

# public access (static website)
aws s3 website s3://my-app-storage-123 \
  --index-document index.html \
  --error-document error.html
```

DigitalOcean Spaces

```
# via s3cmd (S3-compatible)
s3cmd mb s3://my-app-storage \
  --access_key=$SPACES_KEY \
  --secret_key=$SPACES_SECRET \
  --host=sgp1.digitaloceanspaces.com \
  --host-bucket='%(bucket)s.sgp1.digitaloceanspaces.com'

# upload
s3cmd put ./profile.jpg s3://my-app-storage/uploads/ \
  --acl-public
```

File Upload Handler — Node.js (Express)

```
import express from 'express';
import multer from 'multer';
import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';
```

```ts
import { v4 as uuidv4 } from 'uuid';

const app = express();
const upload = multer({ storage: multer.memoryStorage() });

const s3 = new S3Client({
  region: 'sgp1',
  endpoint: 'https://sgp1.digitaloceanspaces.com',
  credentials: {
    accessKeyId: process.env.SPACES_ACCESS_KEY!,
    secretAccessKey: process.env.SPACES_SECRET_KEY!,
  },
});

app.post('/upload', upload.single('file'), async (req, res) => {
  const file = req.file!;
  const ext = file.originalname.split('.').pop();
  const key = `uploads/${uuidv4()}.${ext}`;

  const command = new PutObjectCommand({
    Bucket: 'my-app-storage',
    Key: key,
    Body: file.buffer,
    ContentType: file.mimetype,
    ACL: 'public-read',
  });

  await s3.send(command);

  const url = `https://my-app-storage.sgp1.digitaloceanspaces.com/${key}`;
  res.json({ url, key });
});

app.listen(3000, () => console.log('API running on :3000'));
```

Upload via HTML Form

```html
<form action="/upload" method="POST" enctype="multipart/form-data">
  <input type="file" name="file" accept="image/*" required>
  <button type="submit">Upload</button>
</form>
```

CDN (Content Delivery Network)

CDN = cache static assets di edge location dekat user → faster load time.

```
User (Jakarta) ──→ Edge (CGK) ── cache hit? ──→ serve from edge
                                  └── cache miss ──→ Origin (SGP) ──→ cache & serve
```

Cloudflare CDN

```
# 1. Daftar Cloudflare, add domain
# 2. Ubah nameserver ke Cloudflare
# 3. Set proxy mode (orange cloud) di DNS → otomatis CDN aktif
```

Pro tip: Pasang Cloudflare di depan DO Spaces / S3 untuk bandwidth gratis + caching.

```
# CNAME record:
cdn.example.com → my-bucket.sgp1.digitaloceanspaces.com
# Set proxy: ON (orange cloud)
```

Output size — Image Optimization

```
import sharp from 'sharp';

// Resize & compress sebelum upload
async function processUpload(buffer: Buffer): Promise<Buffer> {
  return sharp(buffer)
    .resize(800, 800, { fit: 'inside' })
    .jpeg({ quality: 80 })
    .toBuffer();
}

// simpan dengan thumbnail
// key: uploads/xxx.jpg
// key: uploads/thumb/xxx.jpg (200px)
```

Latihan

1. **Deploy Web Server**: Launch Droplet/EC2 (free tier). Install Nginx + Node.js. Deploy simple Express API yang return JSON `{ status: "ok", timestamp }`. Screenshot hasil curl.
2. **Object Storage Upload**: Buat Spaces / S3 bucket. Bikin HTML form + Express handler (contoh di atas) untuk upload file. Upload gambar, pastikan bisa diakses public. Screenshot upload sukses.

3. **CDN Setup**: Register domain (atau pake domain gratis Freenom / nip.io). Arahkan ke Cloudflare. Set proxy mode. Test curl dengan dan tanpa Cloudflare, ukur perbedaan waktu.

```
# tanpa CDN (langsung ke droplet)
curl -w "%{time_total}\n" -o /dev/null -s http://<droplet-ip>/image.jpg

# via CDN
curl -w "%{time_total}\n" -o /dev/null -s https://cdn.example.com/image.jpg
```

4. **Auto Scaling Flow**: Tulis script bash / Terraform yang:
   - Deploy 1 Droplet + install app
   - Buat snapshot
   - Buat Load Balancer
   - Scale out ke 2 instance
   - Cek load balancer akses
   - Catat hasil di README

3. Serverless Functions: Cloudflare Workers, Vercel Edge, Serverless DB & Secrets

Apa Itu Serverless?

Serverless = jalanin kode tanpa ngurus server. Provider yang handle scaling, availability, maintenance. Bayar per eksekusi + per durasi (bukan per jam VM).

Kelebihan

- No server management
- Auto scale to zero (ga bayar kalau ga dipake)
- Scale unlimited (dalam batas provider)
- Lebih murah untuk traffic rendah

Kekurangan

- Cold start (eksekusi pertama lambat)
- Timeout (biasanya 10-30 detik)
- Vendor lock-in
- Debugging lebih susah

Cloudflare Workers

Workers = serverless function jalan di edge network Cloudflare (>330 kota). Runtime: Service Workers API (JavaScript/TypeScript/WASM).

```
User ──→ Edge (Jakarta) ──→ Worker ──→ fetch dari API origin
                                  └── return response langsung
```

Setup

```
# install wrangler CLI
npm create cloudflare@latest my-worker

# atau manual
npm install -g wrangler
wrangler init my-worker
```

Worker Sederhana

```
export default {
  async fetch(request: Request, env: Env, ctx: ExecutionContext): Promise<Respon
    const url = new URL(request.url);
    const name = url.searchParams.get('name') || 'World';

    return new Response(`Hello, ${name}!`, {
      headers: { 'content-type': 'text/plain' },
    });
  },
};
```

Worker dengan R2 (Object Storage)

```
export default {
  async fetch(request: Request, env: Env): Promise<Response> {
    const url = new URL(request.url);
    const key = url.pathname.slice(1);

    if (!key) return new Response('Not Found', { status: 404 });

    const object = await env.MY_BUCKET.get(key);

    if (!object) return new Response('Not Found', { status: 404 });

    return new Response(object.body, {
      headers: {
        'cache-control': 'public, max-age=31536000',
        'content-type': object.httpMetadata?.contentType || 'application/octet-s
      },
    });
```

```
  },
};
```

Deploy

```
wrangler deploy
```

Vercel Edge Functions

Vercel Edge Functions = serverless via Vercel, runtime Edge (V8 isolate). Jalan di 18+ lokasi.

Setup

```
npx create-next-app@latest my-edge-app --typescript
cd my-edge-app
```

Edge API Route (app/api/hello/route.ts)

```
export const runtime = 'edge';

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const name = searchParams.get('name') ?? 'World';

  return new Response(`Hello, ${name} from Edge!`, {
    headers: { 'content-type': 'text/plain' },
  });
}
```

Deploy ke Vercel

```
npx vercel deploy --prod
```

Railway Deploy

Railway = PaaS simpel. Connect GitHub → auto deploy. Support Node.js, Python, Go, Docker.

Quick Deploy — Express API

```
{
  "name": "my-api",
  "scripts": {
```

```
    "start": "tsx src/index.ts"
  }
}
```

```
# deploy via CLI
npm i -g @railway/cli
railway login
railway init
railway up
```

Railway + PostgreSQL

```
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();

// DATABASE_URL otomatis di-set oleh Railway
app.get('/users', async (req, res) => {
  const users = await prisma.user.findMany();
  res.json(users);
});
```

Serverless Database

Turso (Edge SQLite)

Turso = SQLite distributed, lebih cepat dari serverless Postgres untuk read-heavy workload.

```
# install turso CLI
curl -sSfL https://get.turso.tech | bash

# login
turso auth login

# create database
turso db create my-app-db

# get connection string
turso db show my-app-db --url

# create token
turso db tokens create my-app-db
```

```
import { createClient } from '@libsql/client';

const db = createClient({
  url: process.env.TURSO_DATABASE_URL!,
```

```
  authToken: process.env.TURSO_AUTH_TOKEN!,
});

async function getUsers() {
  const result = await db.execute('SELECT * FROM users LIMIT 10');
  return result.rows;
}
```

Neon (Serverless Postgres)

Neon = Postgres serverless dengan branching (seperti git untuk DB).

```
# via CLI
npm i -g @neondatabase/cli

# create project
neon projects create --name my-app
neon connection-string --project-id my-app --branch main
```

```
import { neon } from '@neondatabase/serverless';

const sql = neon(process.env.DATABASE_URL!);

const rows = await sql`SELECT * FROM users LIMIT 10`;
console.log(rows);
```

Perbandingan DB Serverless

Fitur	Turso	Neon	PlanetScale
Engine	SQLite (libSQL)	Postgres	MySQL (Vitess)
Edge ready	✅	✅	✅
Cold start	~5ms	~50ms	~100ms
Branching	✅ (via Turso)	✅ (git-like)	✅
Free tier	9GB storage, 1B rows read/bln	0.5GB storage, 100h compute	1GB storage, 10M row reads
Replication	Embedded replica	Read replicas	Read replicas

Environment Variables & Secrets Management

Jangan Hardcode Secrets!

```
// ❌ JANGAN
const API_KEY = 'sk-12345-abcde';

// ✅ PAKAI environment variables
const API_KEY = process.env.API_KEY!;
```

Cloudflare Workers — `wrangler.toml`

```
name = "my-worker"
main = "src/index.ts"

[vars]
MY_SECRET = "ini-bukan-secret"  # visible di dashboard
```

```
# secret (ga visible)
wrangler secret put API_KEY

# untuk local
echo "API_KEY=sk-xxxx" > .dev.vars
```

Vercel

```
# CLI
vercel env add API_KEY

# dashboard: Settings → Environment Variables
```

Railway

```
# CLI
railway variables --set API_KEY=sk-xxxx

# atau lewat dashboard Variables tab
```

Docker / Node.js

```
# file .env (JANGAN di-commit!)
DATABASE_URL=postgres://...
JWT_SECRET=supersecret
```

```
import dotenv from 'dotenv';
dotenv.config();
```

```ts
export const config = {
  databaseUrl: process.env.DATABASE_URL!,
  jwtSecret: process.env.JWT_SECRET!,
};
```

Best Practice

1. **Jangan commit .env** — tambahin ke `.gitignore`
2. **Gunakan 1Password / Doppler** untuk tim
3. **Rotate secrets** secara berkala
4. **Least privilege** — tiap service pake token sendiri

```
project/.env.local       ← local dev
project/.env.production  ← JANGAN di-commit
project/.env.example     ← template (boleh di-commit)
```

Latihan

1. **Cloudflare Worker API**: Bikin worker yang serve API `/api/weather?city=jakarta`. Return mock data suhu + kondisi. Deploy pake wrangler. Screenshot hasil fetch.

```ts
// response example
{
  "city": "jakarta",
  "temp": 32,
  "condition": "partly cloudy",
  "timestamp": "2026-07-04T10:00:00Z"
}
```

2. **Turso + Worker**: Integrasi Turso DB dengan Cloudflare Worker. Buat endpoint `/users`:
   - `POST /users` — tambah user baru
   - `GET /users` — list users
   - `GET /users/:id` — detail user Test pake curl. Screenshot hasil.
3. **Vercel Edge + Neon**: Deploy Next.js edge function dengan Neon DB. Buat API counter: `GET /api/counter` return `{ count: number }`, `POST /api/counter` increment. Deploy ke Vercel. Screenshot hasil.
4. **Secrets Management**: Bikin script bash yang:
   - Setup `.env.example` dengan template
   - Setup `.gitignore` yang include `.env`
   - Set 3 secrets di Cloudflare Workers pake wrangler
   - Verifikasi secret ga muncul di source code

- Catat langkah-langkah di README

4. IaC & Deploy: Terraform, Docker, CI/CD, Monitoring

Infrastructure as Code (IaC)

IaC = manage infrastructure pakai code (bukan klik-klik UI). Version controlled, repeatable, auditable.

Kenapa IaC?

- **Reproducible** — infra yang sama tiap deploy
- **Version controlled** — track perubahan infra
- **Reviewable** — PR untuk perubahan firewall? Bisa!
- **Automated** — ga ada human error klik salah

Terraform

Terraform = de facto IaC tool. HCL (HashiCorp Configuration Language). Support semua provider.

Install

```
# Linux/WSL
sudo apt-get update && sudo apt-get install -y gnupg software-properties-common
wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor | sudo tee /usr/
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://
sudo apt update && sudo apt install terraform
```

DigitalOcean — Deploy Droplet + Firewall

```
# main.tf
terraform {
  required_providers {
    digitalocean = {
      source = "digitalocean/digitalocean"
      version = "~> 2.0"
    }
  }
}

provider "digitalocean" {
```

```
  token = var.do_token
}

resource "digitalocean_droplet" "web" {
  image    = "ubuntu-24-04-x64"
  name     = "web-server"
  region   = "sgp1"
  size     = "s-1vcpu-1gb"
  ssh_keys = [data.digitalocean_ssh_key.main.id]

  user_data = <<-EOF
    #cloud-config
    packages:
      - nginx
    runcmd:
      - systemctl enable nginx
      - systemctl start nginx
  EOF
}

resource "digitalocean_firewall" "web" {
  name = "web-firewall"

  droplet_ids = [digitalocean_droplet.web.id]

  inbound_rule {
    protocol         = "tcp"
    port_range       = "22"
    source_addresses = [var.my_ip]
  }

  inbound_rule {
    protocol         = "tcp"
    port_range       = "80"
    source_addresses = ["0.0.0.0/0", "::/0"]
  }

  inbound_rule {
    protocol         = "tcp"
    port_range       = "443"
    source_addresses = ["0.0.0.0/0", "::/0"]
  }

  outbound_rule {
    protocol              = "tcp"
    port_range            = "1-65535"
```

```
    destination_addresses = ["0.0.0.0/0", "::/0"]
  }
}

data "digitalocean_ssh_key" "main" {
  name = "my-key"
}

variable "do_token" {
  type = string
}

variable "my_ip" {
  type = string
}

output "droplet_ip" {
  value = digitalocean_droplet.web.ipv4_address
}
```

```
# Run
terraform init
terraform plan -var="do_token=$DO_TOKEN" -var="my_ip=$(curl -s ifconfig.me)/32"
terraform apply -var="do_token=$DO_TOKEN" -var="my_ip=$(curl -s ifconfig.me)/32"
terraform destroy  # cleanup!
```

Pulumi

Pulumi = IaC pakai programming language beneran (TypeScript, Python, Go, C#).

```
import * as digitalocean from "@pulumi/digitalocean";

const droplet = new digitalocean.Droplet("web", {
  image: "ubuntu-24-04-x64",
  region: "sgp1",
  size: "s-1vcpu-1gb",
  userData: `#cloud-config
packages:
  - nginx
runcmd:
  - systemctl enable nginx
  - systemctl start nginx`,
});

export const ip = droplet.ipv4Address;
```

```
pulumi up
```

Docker + Cloud

Dockerfile untuk Production

```dockerfile
# Gunakan multi-stage build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY package*.json ./

EXPOSE 3000
CMD ["node", "dist/index.js"]
```

Deploy Docker ke DigitalOcean App Platform

```yaml
# .do/app.yaml
name: my-api
region: sgp1
services:
  - name: api
    http_port: 3000
    image:
      registry_type: DOCKER_HUB
      registry: username/my-api
      registry_credentials: false
    instance_count: 1
    instance_size_slug: basic-xxs
    envs:
      - key: DATABASE_URL
        value: ${DATABASE_URL}
        type: SECRET
```

CI/CD ke Cloud (GitHub Actions)

Deploy ke DigitalOcean Droplet via SSH

```yaml
# .github/workflows/deploy.yml
name: Deploy to DO

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install dependencies
        run: npm ci

      - name: Build
        run: npm run build

      - name: Deploy to Droplet via SSH
        uses: appleboy/scp-action@v0.1.7
        with:
          host: ${{ secrets.DO_HOST }}
          username: root
          key: ${{ secrets.DO_SSH_KEY }}
          source: "dist/,package.json,package-lock.json"
          target: "/root/app"

      - name: Restart app
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${{ secrets.DO_HOST }}
          username: root
          key: ${{ secrets.DO_SSH_KEY }}
          script: |
            cd /root/app
            npm ci --production
```

```
            pm2 restart api || pm2 start dist/index.js --name api
```

Deploy ke Cloudflare Workers

```yaml
# .github/workflows/deploy-worker.yml
name: Deploy Worker

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install dependencies
        run: npm ci

      - name: Deploy to Cloudflare Workers
        uses: cloudflare/wrangler-action@v3
        with:
          apiToken: ${{ secrets.CF_API_TOKEN }}
```

Deploy Docker ke Railway

```yaml
# .github/workflows/deploy-railway.yml
name: Deploy to Railway

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```yaml
      - name: Install Railway
        run: npm i -g @railway/cli

      - name: Deploy
        run: railway up --service my-api
        env:
          RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN }}
```

Monitoring

Uptime Monitoring

```bash
# Simple ping script
while true; do
  status=$(curl -s -o /dev/null -w "%{http_code}" https://your-app.com/health)
  echo "$(date) - Status: $status"
  if [ "$status" != "200" ]; then
    echo "ALERT: App down!"
    # send notification (telegram, email, slack)
  fi
  sleep 60
done
```

BetterStack / UptimeRobot

- **BetterStack**: 5 monitor gratis, check 30 detik, status page
- **UptimeRobot**: 50 monitor gratis, check 5 menit
- **Checkly**: Browser check + API check (bisa pake Playwright)

Logging & Metrics

```bash
# simple log monitoring — journalctl
journalctl -u nginx --since "1 hour ago" --no-pager | grep error

# tail log app
pm2 logs my-api

# structured logging (better)
```

```javascript
// Winston logger — recommended
import winston from 'winston';

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.json(),
  transports: [
```

```
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' }),
  ],
});

// di production, kirim ke: Axiom, Datadog, Grafana, atau BetterStack
```

Health Check Endpoint

```
app.get('/health', async (req, res) => {
  const dbOk = await checkDatabase();
  res.status(dbOk ? 200 : 503).json({
    status: dbOk ? 'healthy' : 'degraded',
    uptime: process.uptime(),
    timestamp: new Date().toISOString(),
  });
});
```

Output Akhir Sesi

Di sesi ini kamu harus punya: - □ Terraform script untuk deploy infra - □ Dockerfile untuk app - □ GitHub Actions workflow deploy otomatis - □ Health check endpoint - □ Monitoring sederhana

Latihan

1. **Terraform Deploy**: Pakai Terraform untuk deploy 1 Droplet + firewall + SSH key. App nginx auto terinstall via user_data atau cloud-config. Output IP droplet. Screenshot `terraform apply` sukses.
2. **Dockerize App**: Buat Dockerfile (multi-stage) untuk API dari sesi 2. Build image, run lokal, test. Push ke Docker Hub atau GitHub Container Registry. Dokumentasi step.
3. **CI/CD Pipeline**: Setup GitHub Actions workflow yang:
   - Build app
   - Jalankan test
   - Deploy ke Cloudflare Workers (free tier)
   - Trigger tiap push ke main Screenshot workflow sukses di GitHub Actions tab.
4. **Full Monitoring**: Bikin health check endpoint (`/health`). Setelah deploy, setup BetterStack atau UptimeRobot monitor. Cek status page. Simulasikan down (stop app), pastikan alert terkirim. Catat response time selama 1 jam.